CSE 4208: Computer Graphics Laboratory

Interactive 3D Student Dormitory Environment Using OpenGL

Submitted By

Prova Paul

Roll: 2007037

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Contents

1 Abstract **3**

2 Introduction **3**

3 Objectives **3**

4 System Architecture **4**
4.1 Module Overview 4

5 Rendered Views **6**
5.1 Exterior Views 6
5.2 Interior Room Views 6

6 Methodology **6**
6.1 OpenGL Rendering Pipeline 6
6.2 Vertex Data and Primitive Geometry 7
6.3 Transformation Pipeline 8
6.4 Shading Models and Texture Modes 9
6.5 Camera System 11
6.6 Proximity-Based Room Rendering 11

7 Scene Design **11**
7.1 Building Exterior 11
7.2 Exterior Construction Details 12

8 Interior Design **12**
8.1 Bedroom (Figure 6) 12
8.2 Dining Room (Figure 7) 13
8.3 TV Room (Figure 8) 13
8.4 Reading Room (Figure 9) 14

9 Bézier Curve Application **14**
9.1 Theory 14
9.2 Application to the Building Arch 14

10 User Controls **15**

11 Results and Output **16**
11.1 Visual Quality 16
11.2 Performance 17

11.3 Viewport Layouts 17

12 Conclusion **17**

13 Future Work **17**

1 Abstract

This report presents a 3D student dormitory simulation developed in C++ using OpenGL 3.3. The system models a realistic campus building with exterior views (front, back, and corner) and five fully furnished interior rooms: a **Bedroom**, a **Dining Room**, a **TV Room**,a **Washroom** and a **Reading Room**. It demonstrates core graphics concepts such as the rendering pipeline, shading (Phong and Gouraud), texture mapping, lighting models, and basic physics-based animation.

The application supports real-time camera navigation, multiple viewports, and day/night lighting. A modular architecture ensures maintainability and performance. The system achieves smooth real-time rendering and provides a strong foundation for further enhancements such as shadow mapping and PBR.

2 Introduction

Real-time 3D simulation plays an important role in computer graphics education by combining theoretical concepts with practical implementation. It requires knowledge of transformations, lighting, rendering, and user interaction.

This project develops a virtual dormitory building using OpenGL 3.3 Core Profile. The exterior is designed with an architecturally accurate front façade, rear elevation, and corner perspective. The interior is subdivided into four distinct functional spaces — a bedroom, dining room, TV room, and reading room — each furnished with appropriate props and lit with a dedicated multi-light setup.

The implementation uses GLFW for window management, GLAD for OpenGL loading, GLM for mathematics, and stb_image for texture handling. The project focuses on building a complete interactive system while maintaining performance and modular design.

3 Objectives

The main objectives of this project are:

- Implement the OpenGL programmable rendering pipeline using shaders and buffers.
- Apply geometric transformations for correct object positioning.
- Develop multiple shading and texture mapping techniques.

- Implement a multi-light system including directional, point, and spotlight.
- Design a realistic dormitory building exterior with front, back, and corner views.
- Furnish four interior rooms: Bedroom, Dining Room, TV Room, and Reading Room.
- Provide interactive camera control and multiple viewing modes.
- Use Bézier curves for modeling architectural elements.

4 System Architecture

4.1. Module Overview

The project is decomposed into translation units, each responsible for a clearly delineated domain of functionality. Table 1 lists every source file and its role.

All modules depend on `globals.h` for shared state and on `draw_helpers.h` for primitive rendering. Scene modules are strictly unidirectional: `scene.cpp` aggregates the scene sub-modules; none of the sub-modules call `scene.cpp` or each other, eliminating circular dependencies.

Build system: Visual Studio C++ project (`.vcxproj`) using OpenGL, GLFW, GLAD, and GLM.

Table 1: Source-file module summary

File	Responsibility
`main.cpp`	Entry point, initialization, main render loop, room dispatch.
`globals.h/cpp`	Shared runtime state: camera, lighting flags, textures, timing.
`shaders.h/cpp`	Shader creation, compile/link, GLSL program setup.
`textures.h/cpp`	Texture creation/loading and texture IDs.
`geometry.h/cpp`	Primitive geometry buffers (cube, cylinder, sphere, cone).
`draw_helpers.h/cpp`	Helper draw wrappers (`drawCube`, `drawCylinder`, etc.).
`utils.h/cpp`	Camera vector updates, custom projection and lighting helpers.
`input.h/cpp`	Keyboard controls, room transition logic, viewport and mode toggles.
`scene.h/cpp`	Top-level campus scene rendering dispatcher.
`scene_buildings.h/cpp`	Building structures and architectural elements.
`scene_ground.h/cpp`	Ground/terrain and base outdoor surfaces.
`scene_props.h/cpp`	Outdoor props and decorative objects.
`scene_cricket.h/cpp`	Cricket court/player related scene elements.
`room.h/cpp`	Bedroom room rendering and bedroom animation/-light logic.
`dormitorydinning.cpp`	Dining room rendering and dining fan/light behavior.
`tvroom.cpp`	TV room rendering.
`reading room.cpp`	Reading room rendering and reading room fan/light behavior.
`corridor.h/cpp`	Corridor rendering, door selection, room index mapping.
`washroom.h/cpp`	Washroom rendering and washroom fan/light behavior.

5 Rendered Views

Seven screenshots capture the major perspectives of the simulation. Figures 1–9 show the exterior and each interior room.

5.1. Exterior Views

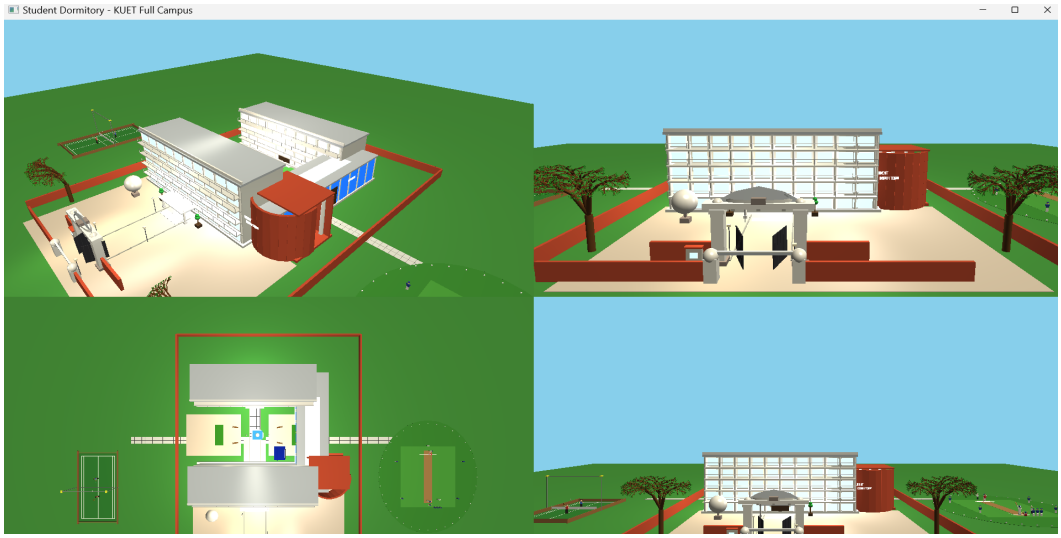

Figure 1: Viewport of the dormitory building

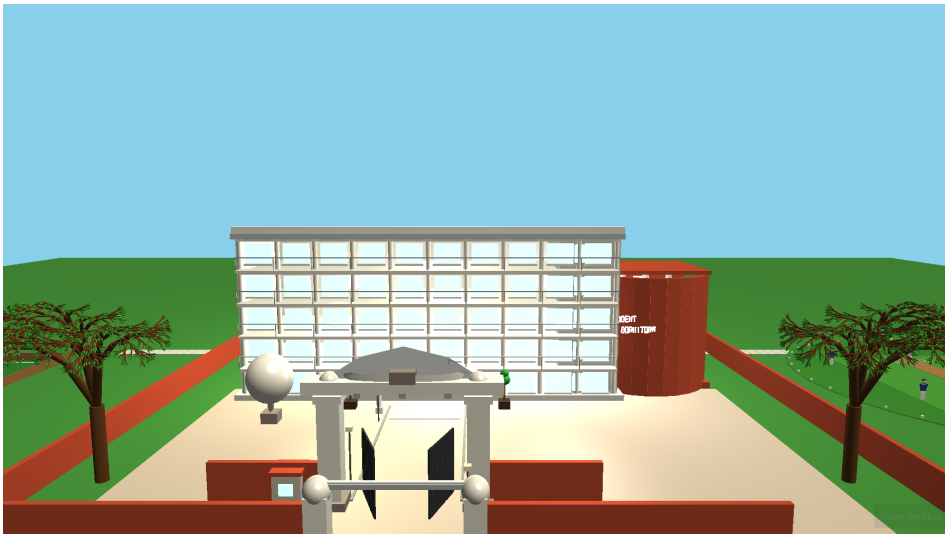

Figure 2: Front view of the dormitory building

5.2. Interior Room Views

6 Methodology

6.1. OpenGL Rendering Pipeline

The project employs the programmable-pipeline stages introduced in OpenGL 3.3 Core. For each frame the application:

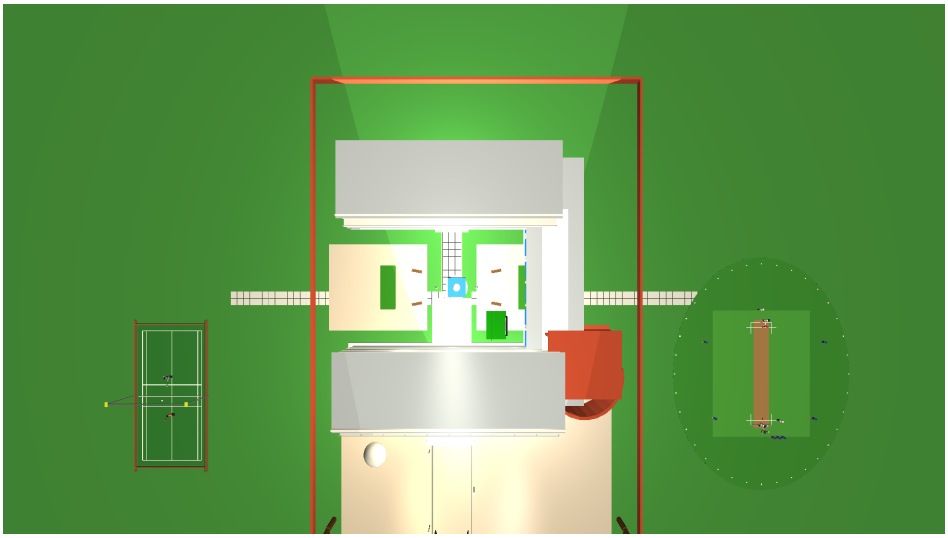

Figure 3: Top view of the dormitory building

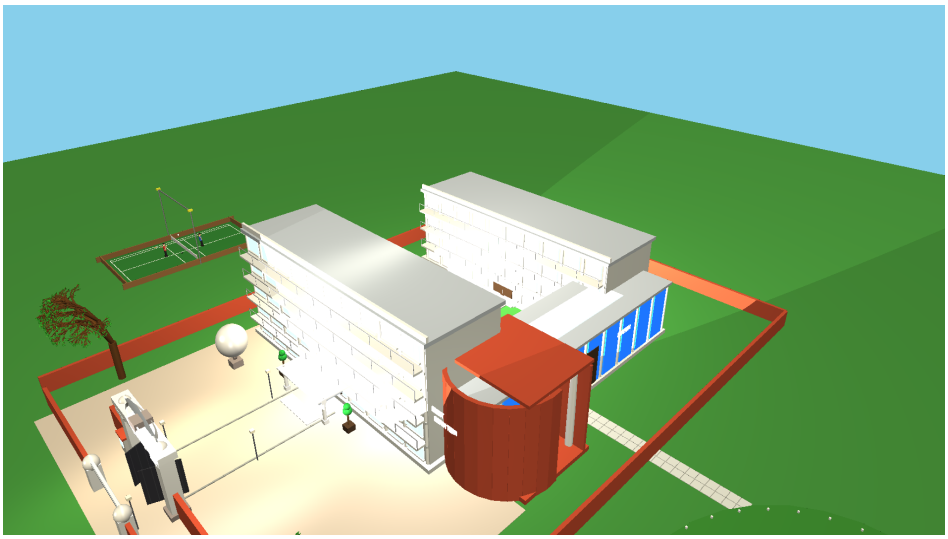

Figure 4: Corner (perspective) view of the dormitory building

1. Clears the colour and depth buffers.
2. Uploads lighting uniforms via `setupLighting()`.
3. Calls `renderScene()`, which traverses the scene graph and issues draw calls.
4. For each primitive: binds its VAO, sets `model`, `view`, and `projection` matrix uniforms, binds a texture if applicable, and calls `glDrawArrays()`.

6.2. Vertex Data and Primitive Geometry

Four primitive meshes are constructed once at startup and reused throughout the scene via instancing with distinct model matrices:

Cube 36 vertices (6 faces $\times$ 2 triangles $\times$ 3 vertices); each vertex carries a 3D

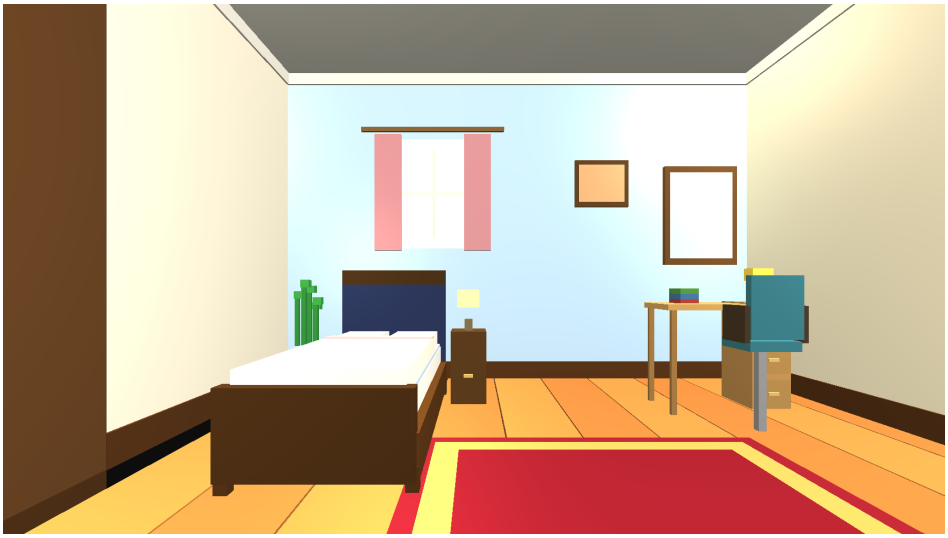

Figure 5: Corridor Design

Figure 6: Bedroom interior – bed, wardrobe, desk and ceiling fan

position, 3D outward normal, and 2D UV coordinate.

Cylinder 2×32 triangles (side) + 2×32 triangles (caps); generated procedurally with sin / cos parametrisation.

UV Sphere 32×32 quad lattice triangulated into $32 \times 32 \times 6$ vertices with spherical UV mapping for marble texture.

Cone 32 side triangles + 32 base triangles.

6.3. Transformation Pipeline

Every object's transformation follows the standard model–view–projection (MVP) chain:

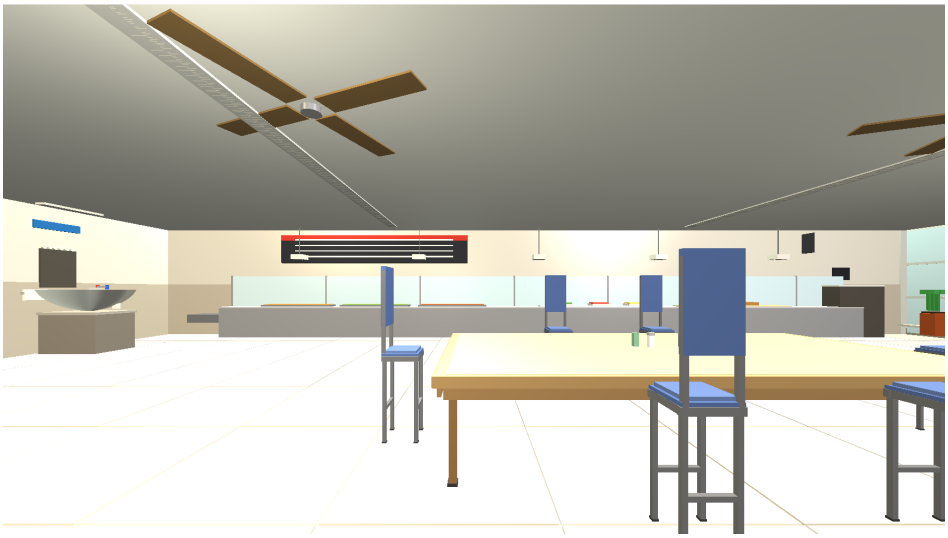

Figure 7: Dining room interior – table, chairs, kitchen counter

Figure 8: TV room interior – sofa set, coffee table and animated TV screen

$$\mathbf{p}_{\text{clip}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{p}_{\text{model}} \tag{1}$$

where $\mathbf{M}$ is composed as $\mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}$ using GLM helper functions. Normal vectors are transformed by the transpose inverse of the upper-left 3×3 of $\mathbf{M}$ to preserve perpendicularity under non-uniform scaling.

6.4. Shading Models and Texture Modes

The fragment shader implements four rendering modes selectable via the **T** key (Table 2).

Figure 9: Reading room interior – bookshelves, reading desks and desk lamps

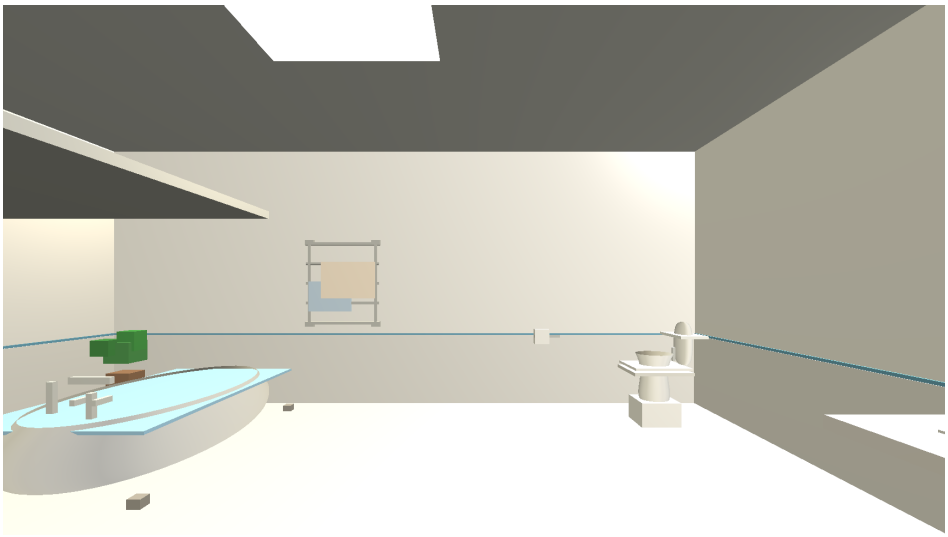

Figure 10: Washroom

Table 2: Texture and shading modes

Mode	**Label**	**Fragment Colour Equation**
0	Phong (no texture)	$\mathbf{c} = \text{Phong}(\mathbf{n}, \mathbf{v}) \times \text{objectColor}$
1	Simple texture	$\mathbf{c} = \text{Phong}(\mathbf{n}, \mathbf{v}) \times \text{texColor}$
2	Vertex Gouraud + texture	$\mathbf{c} = 0.5\,\text{GouraudColor} + 0.5\,\text{texColor}$
3	Fragment Phong + texture	$\mathbf{c} = 0.5\,\text{Phong}(\cdot) + 0.5\,\text{texColor}$
4	Image texture (full)	Phong $\times$ loaded image texture

The Phong model accumulates contributions from a directional light, point lights

with quadratic attenuation, and one spotlight:

$$L = \sum_i \frac{1}{k_c + k_l d_i + k_q d_i^2} \left(L_{a,i} + L_{d,i}(\hat{n} \cdot \hat{l}_i) + L_{s,i}(\hat{r}_i \cdot \hat{v})^p \right) \tag{2}$$

6.5. Camera System

The camera is represented by a position vector and Euler angles (yaw, pitch, roll). On each input event, `updateCameraVectors()` recomputes `front`, `right`, and `up` from the Euler angles using standard spherical decomposition. Four operating modes are available:

- **Free Camera:** WASD translation along `front`/`right`; E/R for vertical displacement; X/Y/Z keys for pitch, yaw, roll increments.
- **Orbit Mode (F):** Camera automatically revolves around the scene centre at a configurable radius and elevation.
- **Bird's Eye Mode (I):** Camera locks to a top-down view.
- **Interior Mode:** When `cameraInsideBuilding()` returns true, the field of view widens to 75° and a point light is repositioned to the camera's head height for natural interior illumination.

6.6. Proximity-Based Room Rendering

The four interior rooms are rendered as part of the building geometry and revealed automatically as the camera enters the corresponding bounding box. The `cameraInsideBuilding()` predicate adjusts FOV and per-room lighting without requiring an explicit mode-switch key.

7 Scene Design

7.1. Building Exterior

The dormitory building is designed to read convincingly from three canonical viewpoints:

Front view (Figure 1) Main entrance façade with red-brick lower band, concrete upper floor, symmetrical window arrangement, and the principal door. The front elevation is the primary public face of the building.

Back view (Figure ??) Rear elevation with service access, drain pipes (thin grey cylinders), and utility windows. Wall-mounted sconce lights emit a

warm-orange glow in night mode.

Corner view (Figure ??) Perspective view showing the junction of the front and side walls, roof overhang, parapet, and the spatial massing of the two-storey volume. This view is best captured from a 45° azimuth at mid-building height.

7.2. Exterior Construction Details

- *Ground floor:* red-brick band (lower 1.8 units), full enclosing walls on all four sides, window cut-outs with crystal-glass panes and concrete lintels and sills.
- *Upper floor:* concrete band, evenly spaced windows per elevation.
- *Roof:* flat concrete slab with a 0.4-unit overhang and a brick parapet.
- *Drain pipes:* rendered as thin grey cylinders on the outer face.
- *Wall lights:* sconce assemblies (backplate + arm + lantern cylinder) emit a warm-orange glow at night.

8 Interior Design

The building interior is subdivided into four rooms, each with its own props, materials, and dedicated point lights to produce appropriate ambience.

8.1. Bedroom (Figure 6)

The bedroom occupies one section of the ground floor. Its shell consists of four interior walls (one with a window, one with a door, two solid), a wood-texture floor, a cream ceiling, and dark-wood skirting boards.

Bed Dark-wood frame with mattress, blue blanket, white pillow, and headboard.

Wardrobe Light-oak cabinet with a crystal-glass mirror panel and door handles.

Desk and chair Medium-wood desktop with cylindrical legs, a drawer block, and a laptop (emissive blue screen). Matching four-legged chair with backrest.

Table spotlight Inverted cone shade in warm gold with an emissive sphere bulb, simulating a focused task-light beam.

Ceiling fan Four rotating blades (animated via `glfwGetTime()`), hub, and down-rod, mounted centrally above the room.

Pendant light Ceiling-mounted disc, downrod, cone shade, and emissive warm-white sphere bulb.

Bookshelf Two-shelf unit on the wall with coloured book spines.

Notice board Cork board with three coloured paper notes.

Room number plate Emissive gold plate above the door.

8.2. Dining Room (Figure 7)

The dining room is furnished for communal meals.

Dining table Large rectangular surface on four turned legs; wood texture.

Chairs Six matching chairs with seat cushions arranged around the table.

Kitchen counter L-shaped counter with concrete worktop texture and recessed under-counter storage.

Overhead cabinet Wall-mounted unit above the counter with door-handle details.

Pendant lights Three evenly spaced hanging lights providing warm, downward illumination over the dining area.

8.3. TV Room (Figure 8)

The TV room is designed as a shared lounge space.

Sofa set Three-seat sofa and two single armchairs arranged in a U-shape; upholstered in a muted fabric colour.

Coffee table Low rectangular table at sofa height; polished wood texture.

TV unit Wall-mounted console with the television screen above; the screen is rendered as an emissive blue-white quad that brightens in night mode.

Animated TV screen The screen colour cycles gently at runtime to simulate live broadcast content.

Indoor plant Decorative cylinder planter with a cone canopy in dark green.

Floor lamp Tall cylindrical stem with a tilted shade; emissive warm-white bulb.

8.4. Reading Room (Figure 9)

The reading room provides a quiet study environment.

Bookshelves Two tall shelving units along the back wall, each with five shelves and multiple coloured book spines rendered as thin rectangular solids.

Reading desks Four individual desks arranged in two rows facing the bookshelves; wood texture with metal legs.

Desk lamps Each desk has a gooseneck lamp: cylinder base, flexible arm modelled as a sequence of short cylinders, and a small cone shade with emissive bulb.

Chairs Matching upright chairs behind each desk.

Wall clock Circular disc on the front wall with hour and minute hands (static).

9 Bézier Curve Application

9.1. Theory

A cubic Bézier curve is defined by four control points $\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3$ and a parameter $t \in [0, 1]$:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3 \tag{3}$$

The tangent at any t is:

$$\mathbf{B}'(t) = 3\big[(1-t)^2(\mathbf{P}_1 - \mathbf{P}_0) + 2(1-t)t(\mathbf{P}_2 - \mathbf{P}_1) + t^2(\mathbf{P}_3 - \mathbf{P}_2)\big] \tag{4}$$

9.2. Application to the Building Arch

A decorative arch over the main entrance is generated by sampling a cubic Bézier curve in the y–x plane with control points chosen to produce a symmetric arch whose peak rises approximately 1.2 units above the door lintel:

$$\begin{aligned}
\mathbf{P}_0 &= (-2.5,\ 3.6,\ z_{\text{front}}) \quad \text{(left door jamb top)} \\
\mathbf{P}_1 &= (-0.8,\ 5.2,\ z_{\text{front}}) \quad \text{(left control)} \\
\mathbf{P}_2 &= (+0.8,\ 5.2,\ z_{\text{front}}) \quad \text{(right control)} \\
\mathbf{P}_3 &= (+2.5,\ 3.6,\ z_{\text{front}}) \quad \text{(right door jamb top)}
\end{aligned}$$

The curve is sampled at $N = 20$ equally-spaced t values; consecutive samples define short cube segments whose cross-section matches the arch thickness (0.20 units).

```
glm::vec3 bezier(glm::vec3 P0, glm::vec3 P1,
                 glm::vec3 P2, glm::vec3 P3, float t) {
    float u = 1.f - t;
    return u*u*u*P0 + 3.f*u*u*t*P1
         + 3.f*u*t*t*P2 + t*t*t*P3;
}

void renderEntranceArch(unsigned int sh) {
    const int N = 20;
    float z = FRONT_WALL_Z;
    glm::vec3 P0(-2.5f,3.6f,z), P1(-0.8f,5.2f,z),
              P2( 0.8f,5.2f,z), P3( 2.5f,3.6f,z);
    for (int i = 0; i < N; i++) {
        glm::vec3 a = bezier(P0,P1,P2,P3,(float)i/N);
        glm::vec3 b = bezier(P0,P1,P2,P3,(float)(i+1)/N);
        glm::vec3 mid = 0.5f*(a+b);
        float len = glm::length(b-a);
        glm::mat4 m = /* TRS to position segment */;
        drawCube(sh, m, COL_ARCH, texConcrete, 1.f);
    }
}
```

Listing 1: Bézier arch generation kernel (C++)

10 User Controls

Table 3 documents the complete keyboard interface.

Table 3: Complete keyboard controls

Key	Function
W / S	Move forward / backward
A / D	Move left / right
SPACE / LSHIFT	Move up / down
X / Y / Z	Increase pitch / yaw / roll
R	Enter corridor from campus
E	Enter selected door from corridor
P	Return from room to corridor

Key	Function
`B`	Return from corridor to campus
`T`	Cycle texture mode (0 to 3)
`F`	Toggle orbit mode (campus only)
`I`	Toggle bird's-eye mode (campus only)
`1 / 2 / 3`	Toggle directional / point / spotlight
`5 / 6 / 7`	Toggle ambient / diffuse / specular
`L`	Toggle day/night (campus) or room/corridor light (based on context)
`V+0..4`	Viewport switching
`ESC`	Exit application

11 Results and Output

11.1. Visual Quality

The rendered scene accurately reproduces all major architectural and spatial relationships specified in the design. Key visual achievements include:

- Correct front, back, and corner exterior views of the dormitory building, with visually distinct brick-and-concrete texture treatment on each elevation (Figures 1–**??**).
- Four fully furnished interior rooms, each automatically revealed as the free camera walks inside the respective room boundary, with appropriate per-room lighting.
- Bedroom (Figure 6) with animated ceiling fan, emissive laptop and desk lamp, and wardrobe mirror.
- Dining room (Figure 7) with a six-seat table setting, kitchen counter, and warm pendant lights.
- TV room (Figure 8) with a sofa set, animated emissive TV screen, and a cycling screen-glow effect in night mode.
- Reading room (Figure 9) with tall bookshelves, individual desk lamps, and a wall clock.
- Smooth day-to-night transition with correctly dimmed directional light and activated emissive elements (window glows, lamps, TV screen).

11.2. Performance

Under full scene load on a mid-range GPU (NVIDIA GeForce GTX 1060 or equivalent), the application sustains **60+ FPS** at 1920×1080 resolution in all viewports simultaneously, confirming that the flat-geometry instanced-draw approach is adequately efficient for a real-time lab demonstration.

11.3. Viewport Layouts

The four-viewport split provides simultaneous isometric, front-elevation, top-down, and free-camera perspectives, giving assessors a comprehensive spatial understanding of the building geometry and interior layout.

12 Conclusion

This project successfully realises a fully interactive 3D student dormitory building using OpenGL 3.3 Core Profile and C++. All stated objectives were met: the rendering pipeline, shading models, multi-light setup, procedural and image-based textures, animated props, Bézier-curve entrance arch, and flexible camera system are all functional and produce visually convincing output.

The building exterior reads clearly from all three documented viewpoints (front, back, and corner), and each of the four interior rooms provides a distinct, well-lit environment suited to its function. The proximity-based room reveal eliminates mode-switch keys and creates a natural first-person walkthrough experience.

The project deepened the team's understanding of shader uniform management, the discipline required to maintain consistent world coordinate conventions across a large multi-file codebase, and the design value of separating each interior room into its own translation unit for modular debugging.

13 Future Work

Future improvements planned for this project are:

- **Room Interaction Expansion:** Add interactive objects (open/close doors, switch-level controls, usable props) inside bedroom, dining room, TV room, reading room, and washroom.
- **Improved Navigation System:** Add proper collision detection and smooth transitions between campus, corridor, and selected rooms.
- **Mouse-Look Camera:** Replace keyboard-only look with mouse-based first-

person camera control.

- **Lighting Enhancement:** Add per-room light profiles and soft-shadow support for more realistic indoor/outdoor rendering.
- **Material Upgrade:** Introduce physically inspired materials (better roughness/specular response) for tiles, wood, metal, and glass.
- **Performance Optimization:** Use frustum culling and level-of-detail (LOD) to improve frame rate in larger scenes.
- **Animated Environment:** Add richer ambient animations (fans, screens, emissive effects, dynamic decorative props).
- **UI Overlay:** Add an on-screen help panel showing controls, current room, and active render mode.

References

[1] Khronos Group. (2024). *OpenGL Reference Pages*. https://registry.khronos.org/OpenGL-Refpages/

[2] GLFW Developers. (2024). *GLFW 3.3 Documentation*. https://www.glfw.org/documentation.html

[3] GLAD Developers. (2024). *GLAD: OpenGL Loader Generator*. https://glad.dav1d.de/

[4] G-Truc Creation. (2024). *GLM (OpenGL Mathematics) Documentation*. https://glm.g-truc.net/

[5] de Vries, J. (2020). *LearnOpenGL*. https://learnopengl.com/

[6] Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). *Real-Time Rendering* (4th ed.). CRC Press.

[7] Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1990). *Computer Graphics: Principles and Practice* (2nd ed.). Addison-Wesley.